

Модель полуструктурированных данных и языки запросов к XML-документам

Постреляционные базы данных
Лекция №3

Виноградова М.В.
МГТУ им. Н.Э. Баумана

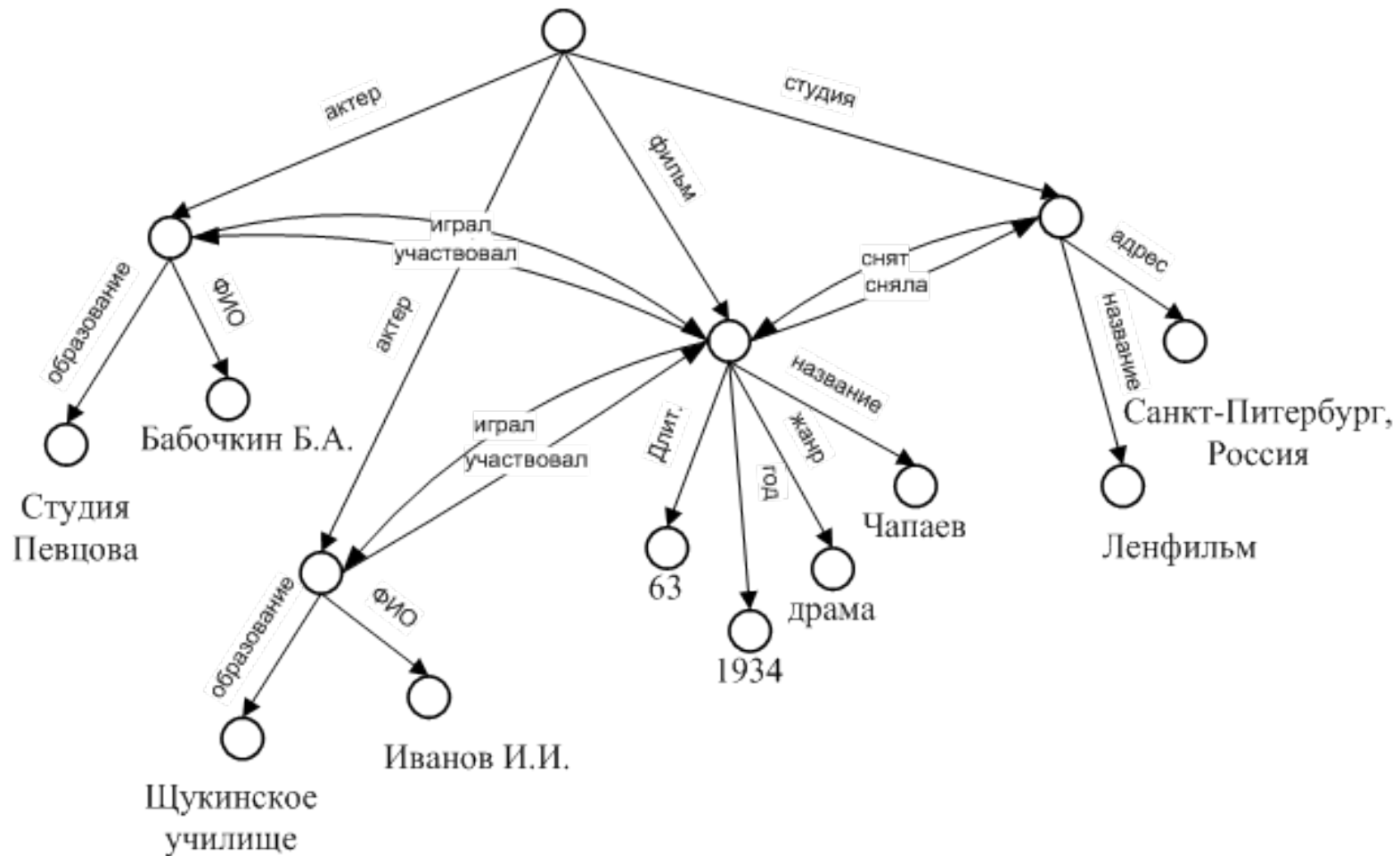
Темы лекции

- Модель полуструктурированных данных
- Граф полуструктурированных данных
- Синтаксис языка XML
- DTD-определения
- Описание полуструктурированных данных посредством XML и DTD
- Язык навигации по XML-документу XPath
- Язык запросов к XML-документу XQuery

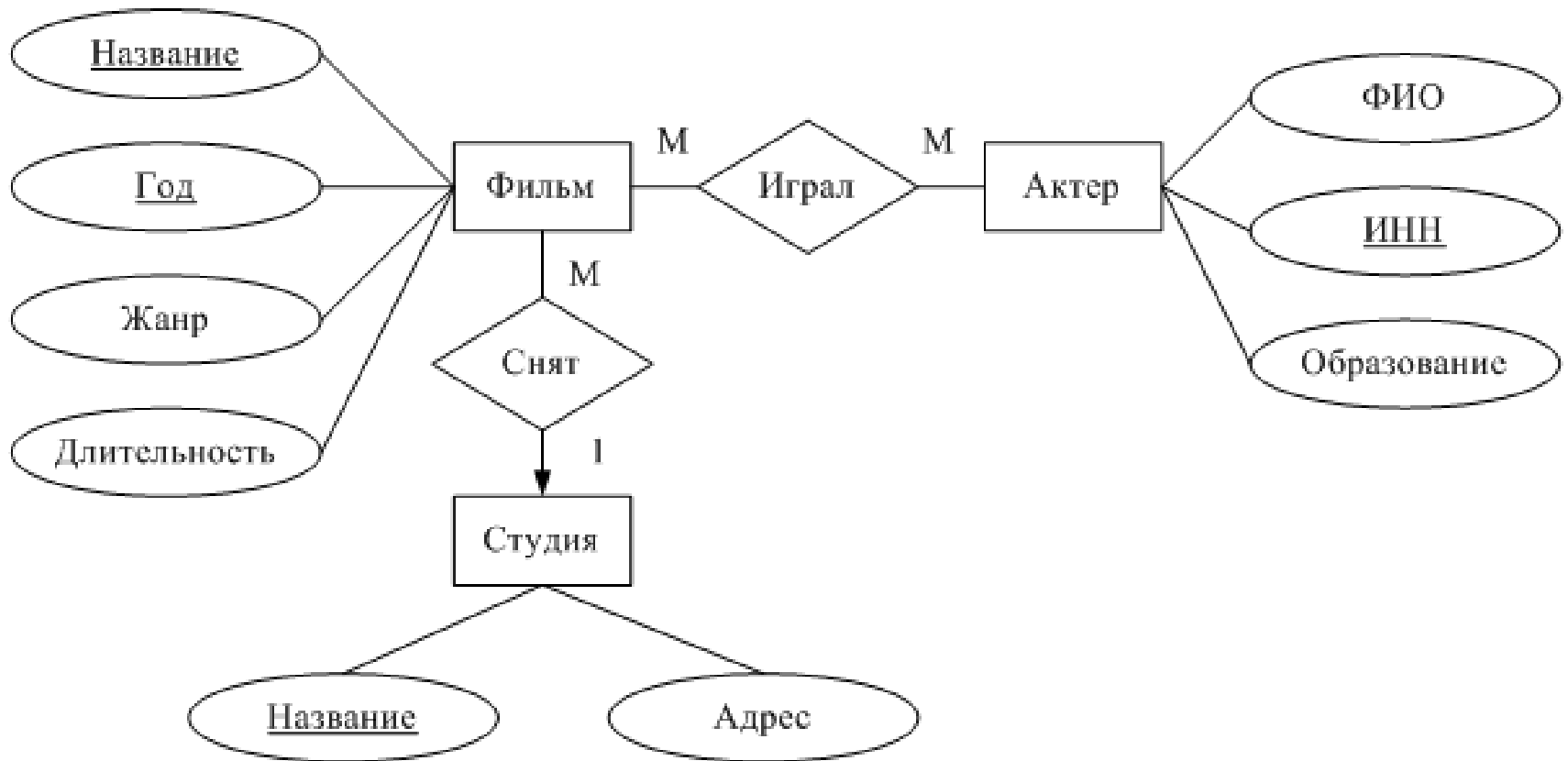
Модель полуструктурированных данных

- Назначение:
 - средство интеграции БД с различными схемами
 - представление документов в инструментальных системах
- Отличия:
 - Нет фиксированной схемы
 - Данные хранят информацию о своей структуре
 - Самопроизвольное изменение схемы (во времени, в контексте БД)

Граф полуструктурированных данных



Пример ER модели



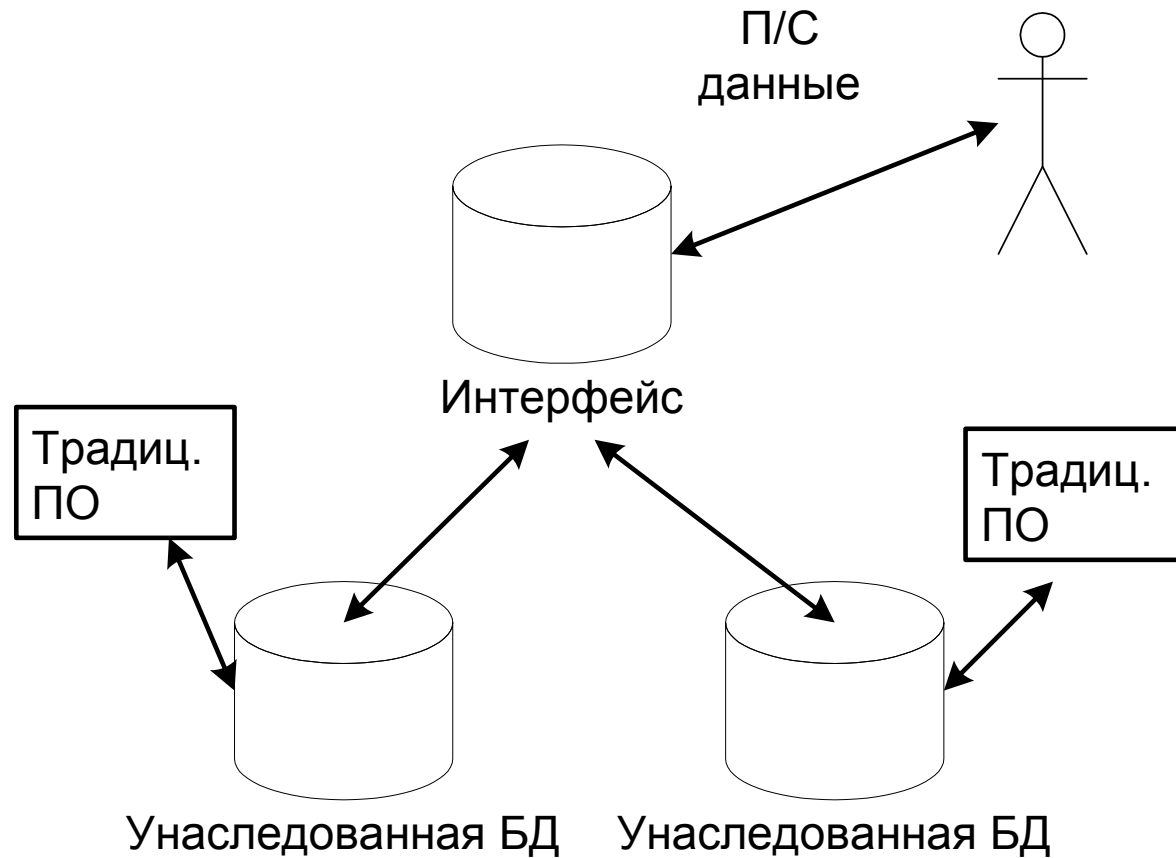
Построение графа

- Узлы:
 - Корень – вся БД
 - Промежуточная вершина – объект или структура
 - Лист – атомарные данные (достигим из корня)
- Не дерево
- Метки дуг:
 - Тип объекта/название структуры/название поля
 - Связи объектов

Преимущества моделей

- БД с фиксированной схемой
 - ❖ эффективность хранения данных
 - ❖ эффективность обработки данных
- Полуструктурированные данные
 - ❖ гибкость
 - ❖ Описание документов
 - ❖ унаследованные данные

Унаследованные БД



Язык XML

- Язык XML (Extensible Markup Language) — расширяемый язык разметки.
- Основан на языке SGML.
- Документ в формате XML содержит текстовые данные и тэги.
- Тэги формируют семантическую разметку документа, определяют смысловое назначение данных.
- Язык XML задает в линейной форме полуструктурированные данные, тэги XML-метки дуг графа полуструктурных данных.

Пример документа XML

```
<?xml version="1.0" standalone="yes"
  encoding=" Windows-1251" ?>
<book>
  <avtor>
    <fio>Текст</fio>
  </avtor>
  <izdanie city="Mos">Еще текст</izdanie>
  <bestbook/>
  <dt year="2000" />
</book>
```

Синтаксис XML - элементы

- Каждый элемент задают тэгом:

`<тэг> содержимое элемента </тэг>`

где `<тэг>` — начальный (открывающий) тэг;

`</тэг>` — завершающий (закрывающий) тэг;

Между ними содержимое элемента.

- Название тэга соответствует названию элемента:

`<fio>Текст</fio>`

- пустой элемент (без содержимого):

`<bestbook/>`

- Вложенность тэгов произвольная, но строго иерархическая.
- Корневой элемент* ровно один на весь документ и включает все прочие элементы. Он обязателен:

`<book> ... </book>`

Синтаксис XML

- *Атрибуты* характеризуют элемент. Атрибуты указывают в открывающем тэге, их значения пишут в одинарных или двойных кавычках (атрибуты city и year):

```
<izdanie city="Mos"> ... </izdanie>  
      <dt year="2000" />
```

- XML-документ начинается строкой:

```
<?xml version="1.0" standalone="yes" encoding=" Windows-1251" ?>
```

где version — версия документа (версии 1.0 или 1.1); standalone — самостоятельность документа (не требуются дополнительные файлы); encoding — кодировка элемента.

- *Комментарии* указывают так:

```
<!-- комментарий -->
```

- Названия тегов и атрибутов регистрозависимы

Синтаксис XML – спец.символы

- Для отображения служебных символов используют их коды:
 - < — знак <;
 - > — знак >;
 - & — знак &;
 - " — знак кавычки;
 - &#код; — символ с указанным кодом.
- Для отображения неформатированного текста используют конструкцию:

```
<![CDATA[ текст ]]>
```
- Для включения инструкций обработки используют конструкцию:

```
<?app .... ?>
```

Правильный и действительный XML-документы

- *Правильный XML-документ (well formed):*
 - содержит любые авторские тэги;
 - не имеет определения схемы;
 - записывает в линейной форме полуструктурированный граф.
- *Действительный XML-документ (valid)*
 - соответствует некоторой схеме, которая задает допустимый набор тэгов, их атрибутов и их вложенность.
 - является средним между жесткой схемой (реляционной БД) и отсутствием схемы (полуструктурированными данными), поскольку в нем могут быть поля, которые не используются.

Язык DTD

- DTD (Document Type Definition) — определение типа документа для автоматизированной обработки XML-документа.
- Документ в формате DTD задает набор допустимых тегов, их атрибутов и правил вложения

Описание элементов

- Каждый элемент XML должен быть описан в DTD конструкцией :
 <!ELEMENT название содержимое>
- Содержимое элемента указывают из следующего списка:
 (#PCDATA) — текст, нет вложенных элементов;
 (el1, el2, el3) — содержит последовательность элементов el1, el2 и el3;
 EMPTY — пустой элемент;
 ANY — любое содержимое.
- Сложная структура вложенных элементов
 - * ноль или более раз;
 - + один или более раз;
 - ? ноль или один раз;
 - | один из указанных элементов.
- Пример элемента E, содержит либо текст, либо набор street и city:
 <!ELEMENT E (# PCDATA | (street*,city*)) >

Описание атрибутов

- Описание атрибута задают конструкцией вида:
 <!ATTLIST элемент атрибут_1 тип параметры
 атрибут_2 тип параметры ... >
- Тип атрибута указывают из следующего списка:
 CDATA — текст;
 (зн1 | зн2 | зн3) — одно из указанных значений;
 ID — уникальное значение (в пределах документа);
 IDREF (или IDREFS) — ссылка на уникальное значение типа ID (ссылки через пробел);
 ENTITY (или ENTITIES) — название сущности/макроса (или несколько через пробел);
 NMTOKEN (или NMTOKENS) — одно слово (набор слов через пробел).
- Параметры атрибутов могут быть следующие:
 #REQUIRED — обязательный;
 #IMPLIED — необязательный
 #FIXED “значение” — фиксированный, имеет указанное значение.
- Описание необязательного атрибута at1 элемента elem (по умолчанию red):
 <!ATTLIST elem at1 (“red”| “green”) #IMPLIED ‘red’ >

Дополнительные описания в DTD

- Пример писания макроса с названием hello:
 <!ENTITY hello system "путь_к_файлу"> — внешняя подстановка (тело в указанном файле);
 <!ENTITY hello "Привет"> — внутренняя подстановка (тело макроса - указанный текст).
- При обращении к макросу перед его именем пишут знак &:
 &hello;
- Для определения макросов в DTD запишем:
 <!ENTITY %имя "текст" >
 <!ENTITY %имя SYSTEM "файл" >
- Приведем пример определения внешнего обработчика. Обработчик nn из файла nn.exe:
 <!NOTATION nn SYSTEM "nn.exe" >
- Подключение обработчика для макроса ff, ссылающегося на данные в файле ff.nn:
 <!ENTITY ff System "ff.nn" NDATA nn>
- Пропишем включение макроса с обработчиком в документ:

Соединение XML и DTD

- **Вариант 1 (внутренний)**
- DTD помещают внутрь XML документа:
<?xml version = “1.0” standalone = “yes” ?>
<!DOCTYPE root [
<!ELEMENT root (elems*) >
...
]**>**
<root>
 <elems> ...</elems>
 <elems> ... </elems>
</root >

Соединение XML и DTD - 2

- **Вариант 2 (внешний)**
- DTD хранят в отдельном файле (с расширением dtd, например, “root. dtd”).
- В XML-документе размещают ссылку на файл DTD:

`<?xml version = “1.0” standalone = “no”?>`

`<!DOCTYPE root SYSTEM “root. dtd”>`

`<root > ... < root >`

- Возможно сочетание внутреннего и внешнего DTD. Доминирует внутренний DTD.

Построение XML документа по графу

- корень (вся БД) — в корневой элемент;
- промежуточная вершина (объект) — во вложенный элемент (метка дуги станет тэгом);
- конечная вершина (атомарное значение) — в содержимое элемента (текст);
- дуга связи между объектами — в набор атрибутов тэга.

Запись в XML иерархии графа

- Если на графе

Вершина n — сущность (структура),
вершина m — его атрибут (поле),

- то в XML помещают вложенные тэги:

<n>

<m> </m>

</n>

Запись в XML связей

- Связь между объектами задают через атрибуты, идентификаторы и ссылки на идентификаторы.
- Если n и m — объекты (вершины) на графе и имеется связь между ними от n к m , то в XML помещают элементы:

`<m mid="m1">....</m>`

`<n tom="m1">....</n>`

- где `mid` – атрибут типа ID (идентификатор объекта m);
- `tom` – атрибут типа IDREF (ссылка на идентификатор объекта m).

Описание связей в DTD

- В DTD-определении записывают:

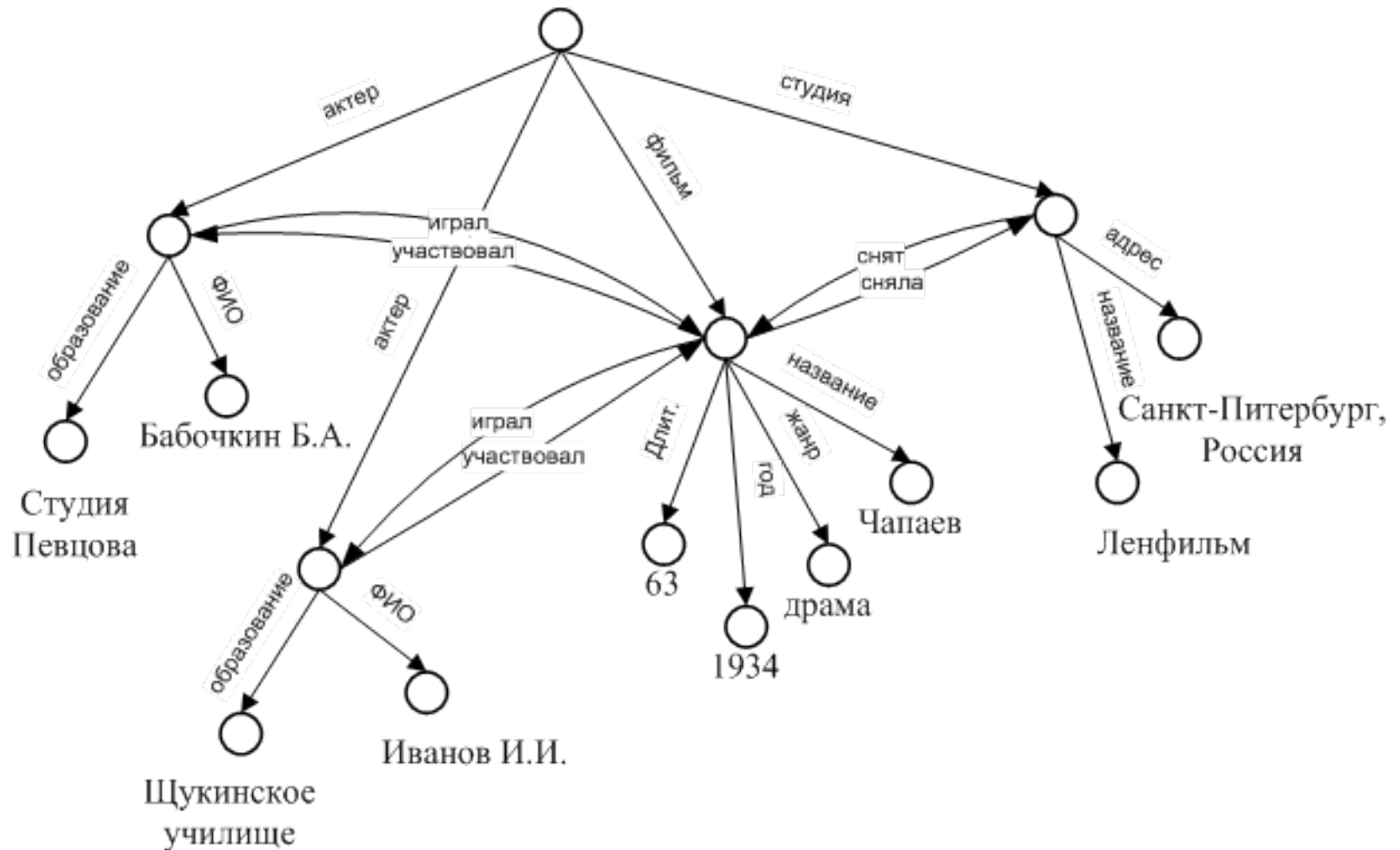
<!ELEMENT n >

<!ELEMENT m>

<!ATTLIST m mid ID>

<!ATTLIST n tom IDREF>

Пример графа



Пример XML

```
<?xml version="1.0" encoding="UTF8" standalone="no"?>
<!DOCTYPE db SYSTEM "file.dtd">
<db>
  <film idf="f1" toact="a1 a2" tos="s1">
    <name>Чапаев</name>
    <year>1934</year>
    <len>63</len>
    <type>драма</type>
  </film>
  <actor ida="a1" tof="f1">
    <fio>Иванов И.И. </fio>
    <edu>Щукинское училище</edu>
  </actor>
  <actor ida="a2" tof="f1">
    <fio>Бабочкин Б.А.</fio>
    <edu>Студия Певцова</edu>
  </actor>
  <studio ids="s1" tof="f1">
    <name>Ленфильм</name>
    <addr>Санкт-Петербург, Россия</addr>
  </studio>
</db>
```

Пример DTD

```
<!ELEMENT db (film*, actor*, studio+)>
<!ELEMENT film (name, year, len?, type?)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT year (#PCDATA)>
<!ELEMENT len (#PCDATA)>
<!ELEMENT type (#PCDATA)>
<!ELEMENT actor (fio, edu+)>
<!ELEMENT fio (#PCDATA)>
<!ELEMENT edu (#PCDATA)>
<!ELEMENT studio (name, addr )>
<!ELEMENT addr (#PCDATA)>
<!ATTLIST film idf ID #REQUIRED toact IDREFS tos IDREF>
<!ATTLIST actor ida ID tof IDREFS>
<!ATTLIST studio ids ID tof IDREFS>
```

XML - Схемы

- АНАЛОГ DTD-определений для описания структуры XML
- составлены по спецификации XSD (XML schema definition)
- XML-схемы имеют следующие преимущества по сравнению с DTD:
 - добавлена работа со стандартными типами данных (int, bool, string и т.д.);
 - позволяют накладывать ограничения на значения (например, год не может быть меньше 1500);
 - сами являются XML-документами;
 - позволяют задавать и использовать пространства имён.
- Схемы более функциональны, чем DTD, но более громоздки и потому удобны для автоматической, а не для ручной обработки документов.

Язык XPath

- Языки XPath и XQuery используют для составления запросов к данным в XML.
- Оба языка разработаны консорциумом W3C.
- Язык XPath (XML Path Language) — язык запросов к элементам XML документа.
- Предназначен для навигации по XML-документу, позволяет указать путь к элементу, условие выборки и извлекаемое значение.

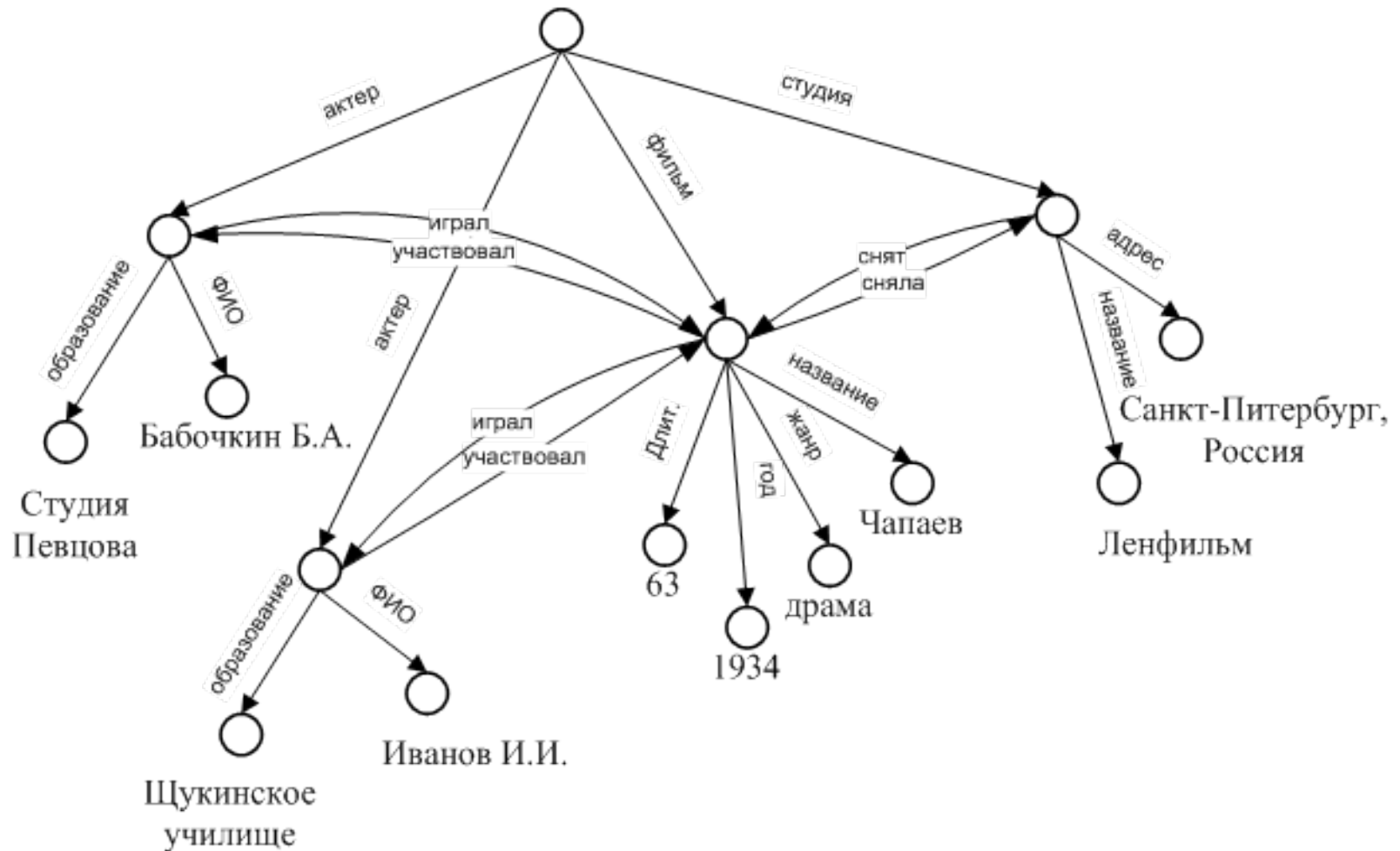
История XPath

- Язык XPath применим для XML-документов любой структуры. Он ориентирован на построение маршрута по графу полуструктурированных данных.
- Версия языка 1.0 имеет максимальную поддержку в браузерах и открытых библиотеках. Версию 2.0 широко используется с 2007г.

Объектная модель документа DOM

- Каждый XML-документ может быть представлен как дерево.
- Элементы и атрибуты документа будут узлами дерева, а отношение вложенности элементов или принадлежности атрибутов — дугами.
- Такое дерево называют объектной моделью документа (DOM — document object model).
- При анализе любого XML-документа строится его объектная модель.
- Язык XPath предназначен для перемещения по дереву и позволяет указать подмножество узлов документа, удовлетворяющих заданному условию.
- Кроме элементов и атрибутов узлами считают комментарии, текстовое содержимое элементов, инструкции, пространства имен.

Пример дерева




```
<?xml version="1.0" encoding="UTF8" standalone="no"?>
<!DOCTYPE db SYSTEM "file.dtd">
<db>
  <film idf="f1" toact="a1 a2" tos="s1">
    <name>Чапаев</name>
    <year>1934</year>
    <len>63</len>
    <type>драма</type>
  </film>
  <actor ida="a1" tof="f1">
    <fio>Иванов И.И. </fio>
    <edu>Щукинское училище</edu>
  </actor>
  <actor ida="a2" tof="f1">
    <fio>Бабочкин Б.А.</fio>
    <edu>Студия Певцова</edu>
  </actor>
  <studio ids="s1" tof="f1">
    <name>Ленфильм</name>
    <addr>Санкт-Петербург, Россия</addr>
  </studio>
</db>
```

Оси документа

- **ancestor** — предки, то есть узлы выше текущего по иерархии вложенности;
- **parent** (родитель) — элемент, в который вложен данный;
- **self** — сам элемент;
- **child** — дети, то есть узлы, непосредственно вложенные в текущий элемент (по умолчанию);
- **descendant** — потомки, то есть узлы ниже текущего по иерархии вложенности;
- **ancestor-or-self** — предки или текущий;
- **descendant-or-self** — потомки или текущий;
- **attribute** — атрибут элемента;
- **namespace** — элемент с пространством имен (с атрибутом xmlns);
- **following** — следующие элементы после текущего по тексту документа, кроме его потомков;
- **following-sibling** — элементы после текущего по тексту документа на его уровне вложенности;
- **preceding** — предыдущие элементы до текущего, кроме его предков;
- **preceding-sibling** — предыдущие элементы на его уровне вложенности.

Путевое выражение

- В запросе на языке XPath указывают направление движения по дереву документа от некоторого узла, направление задается осью.
- **Краткий синтаксис.** Пример путевого выражения для извлечения названия фильма по его идентификатору (атрибуту idf):

`/db/film[@idf="f1"]/name`

- **Полный синтаксис** выглядит так:

`/child::db/child::film[attribute::idf="f1"]/child::name`

Расшифровка путевого выражения

/child::db/child::film[attribute::idf="f1"]/child::name

/child::db/ — шаг первый, от корня документа взять дочерний элемент с названием db,

child::film[attribute::idf="f1"]/ — шаг второй, от полученного результата взять дочерний элемент с названием film, для которого значение атрибута idf равно "f1",

child::name — шаг третий, от полученного результата взять дочерний элемент с названием name

Элементы путевого выражения

- В путевом выражении выделяют:
 - шаги адресации;
 - предикат;
 - условие проверки узлов.
- Шаги адресации задают слева направо и разделяются знаком /
- Шаг содержит:
 - контекст — результат выполнения предыдущего шага (множество узлов). На первом шаге равен корню документа. От него вычисляют новый результат шага;
 - ось (обязательна) — определяет направление движения на шаге;
 - условие (обязательно) — условие выбора узлов по оси: либо имена узлов, либо знак * (все узлы);
 - предикат (необязательный) — фильтр или вычисляемое выражение для фильтрации множества узлов.

Сокращения в путевом выражении

- descendant (потомки) — «//»;
- parent (родитель) — «..»;
- self (сам элемент) — «.»;
- child (дети) — по умолчанию без обозначения;
- attribute — атрибут элемента «@имя».

Пример XPath -1

- Например, запрос вида
/db/actor
- вернет все элементы actor, расположенные внутри элемента db:

```
<actor ida="a1" tof="f1">
```

```
<fio>Иванов И.И. </fio>
```

```
<edu>Щукинское училище</edu>
```

```
</actor>
```

```
<actor ida="a2" tof="f1">
```

```
<fio>Бабочкин Б.А.</fio>
```

```
<edu>Студия Певцова</edu>
```

```
</actor>
```

Примеры XPath - 2

- Запросы к атрибутам возвращают их значения

//film/@idf

вернет значения атрибутов idf всех элементов film, в данном случае это одно значение f1.

- Запросы к содержимому элемента возвращают текстовые значения указанных элементов без их тэгов, например, запрос:

//name/text()

вернет значения:

- Чапаев
- Ленфильм

Запросы к элементам

- /db** — корень (возвращает весь документ);
- /db/film** — все фильмы (возвращает все элементы `<film>..</film>` вместе с их содержимым и тэгами);
- //film** — все фильмы (аналог предыдущего в сокращенном варианте);
- //film/name** — элементы `name` внутри элементов `film`;
- //name** — любые элементы `name` (дочерние для элементов `film` и `studio`);
- //*** — все элементы;
- / */*/name** — элементы `name` 3-ого уровня;
- //actor | // studio** — объединение нескольких путей (актеров и студий);

Запросы к содержимому элемента

//name/text()

— содержимое элементов name;

//fio [. = “Иванов И.И.”]

— элемент fio, значение (текст) которого равно “Иванов И.И.”;

Запросы к элементам с условием

- `//film[1]` — первый элемент `film`;
- `//film[ last ( ) ]` — последний элемент `film`;
- `//fio[ position( ) mod 2 = 0 ]` — каждый второй элемент `fio`;
- `//film[ @idf ]` — все элементы `film`, имеющие атрибут `idf`;
- `//film[ @* ]` — все элементы `film`, имеющие атрибуты;
- `//film[ not ( @* ) ]` — все элементы `film`, не имеющие атрибутов;
- `//film[ ( len or type ) and name ]` — все все элементы `film`, имеющие вложенные элементы `len` или `type` и `name`;

- **//film[@idf = "f1"]** — элементы film, для которых атрибут idf равен f1 (в данном случае один фильм);
- **//actor[count(edu) > 3]** — элементы actor, имеющие более 3-х вложенных элементов edu;
- **//*[start-with(name()), "a"]** — все элементы, названия которых начинаются с "a";
- **//film[3[year="1950"]]** — третий фильм, если он снят в 1950 году;
- **//film[year="1950"][3]** — все фильмы, снятые в 1950, и взять из них третий;

Запросы к атрибутам

- **//*/@***
 - все атрибуты всех элементов (возвращает набор значений);
- **//film/@idf**
 - значения атрибутов idf всех элементов film;
- **//film[year/text()>2000]/@tos**
 - значения атрибутов tos тех элементов film, где вложенный элемент year больше 2000 (идентификаторы всех студий, снявших фильмы после 2000 года);

Запросы на соединение элементов

//studio[@ids = //film[year>2012]/@tos]/name

— названия студий, снявших фильмы после 2012 года (студия, идентификатор которой указана в атрибуте фильма, снятого после 2012, и взять ее название);

//studio[not (@ids = //film[year>2012]/@tos)]/name

— названия студий, не снимавших фильмы после 2012 года (студия, такая, что не существует фильма, снятого после 2012, в атрибуте которого указан идентификатор данной студии, и взять название студии).

Функции языка XPath

Функции работы с множествами:

- **node()** — множество всех узлов в т.ч. текстовых;
- **text()** — текстовый узел (**//text()** — все текстовые узлы);
- **current()** — текущий элемент;
- **last()** — номер последнего элемента в множестве;
- **position()** — номер элемента в множестве;
- **count()** — количество элементов в множестве;
- **name (множество)** — имя первого элемента в множестве;
- **name()** — имя элемента

Логические функции:

- **or** — ИЛИ, **and** — И, **not** — отрицание;
- **=** равно, **!=** не равно;
- **>** больше, **<** меньше, **>=** больше или равно, **<=** меньше или равно;
- **boolean(объект)** — возвращает true или false;
- **true()** — истина; **false()** — ложь.

Приведем примеры условий:

```
[ count (...) > 10 or @x = "..." ]  
[ not (условие ) ]
```


Числовые функции:

- + сумма, - разность, * умножение , **div** деление, **mod** остаток от деления по модулю;
- **sum()** — сумма элементов множества;
- **round()** — округление значения;
- **number(значение)** — приведение значения к числу.

Строковые функции:

- **string**(объект) — приведение к строке;
- **string-length**(строка) — длина строки;
- **contains**(исходная_строка, подстрока) — проверка вхождения подстроки в строку;
- **substring**(строка, начало, количество) — извлечь подстроку начиная с символа (начало) и взять количество символов;
- **starts-with**(строка_исходная, подстрока) — проверка начала строки с подстроки;
- **concat** (s1, s2...) — конкатенация строк;
- **translate**(строкаА, АВ , ав) — замена в 1-ой строке символов из букв 2-й строки на символы из 3-й строки.

Системные функции:

- **document**(объект) — возвращает документ;
- **document**(xml-документ) — возвращает множество узлов;
- **collection** (набор элементов без корня) — возвращает множество узлов.

Язык XQuery

- Язык XQuery — язык запросов к XML-документам, включает в себя язык XPath 2.0.
- Кроме навигации по XML-документу позволяет генерировать новые элементы, определять и выполнять пользовательские функции.
- Содержит конструкции для определения циклов, условий, сортировки, работы с переменными.

Последовательности

- Язык XQuery используют для работы с последовательностями.
- *Последовательность* — упорядоченный набор объектов.
- *Объект* — узел или атомарное значение.
- Последовательности задают так:
1, 2, 3 или **(1, 2, 3)** или **((1, 2), 3)** или **1 to 3**;
() — пустая последовательность.

Элементы XQuery

- *Типы литералов* соответствуют типам XML-схем, например, **xs:string**, **xs:date**, **xs:time** и т.д.
- Значения литералов (констант): 4, 4.7, “hello”, true, false.
Конструктор простого типа: date(“2001–01–01”)
- *Комментарии* записывают так:
{- комментарий -}
- Используют строчные символы для ключевых слов.
- Язык XQuery чувствителен к регистру.
- Вызов *функций* задают как:
имя(параметр)
- Например, вызов функции из базовой библиотеки:
Substring(“hello”, 1, 3)

Путевые выражения

- *Путь* определяется как XPath-выражение.
- Результат любого шага — последовательность узлов, указывают контекст, ось, условие выбора, предикат.
- Поддерживаются краткий и полный синтаксис осей.
- Встроенная функция **document()** возвращает узел — весь документ, далее к нему можно применять путевые выражения, например:

document(“a.xml”)/db/actor[@ida = “a1”]/fio

Пространство имен

- *Пространство имен* дает возможность применять одноименные элементы с различным содержанием и семантикой.
- В языке XML пространство имен задают в тэге с помощью атрибута `xmlns`, например:
<root xmlns = ' http://my.site.ru/mynn.xsd'>
- где `http://my.site.ru/mynn.xsd` — URL-адрес XML-схемы с описанием пространства имен.
- Пространство имен в языке XQuery объявляют так:
declare namespace c= " http://my.site.ru/mynn.xsd"
- где `c` — имя (название) пространства имен.
- Для отнесения элемента или атрибута к конкретному пространству имен его имя записывают перед названием тэга, например:

doc(файл)//c:тег/c:тег2

FLWOR

Основу языка XQuery составляют команды, называемые FLWOR:

- **for** — цикл;
- **let** — присваивание переменной;
- **where** — условие;
- **order by** — сортировка;
- **return** — возврат значений.

Основные конструкции

- Цикл (итерация) записывается следующим образом

for переменная in коллекция

- Присваивание переменной

let переменная = значение

- Возврат значения

return значение

Примеры конструкций

- Переменные записывают как «\$имя», например:

let \$a=10, \$b=15

- Для подстановки вычисленного значения используют { }:

return { \$a + \$b }

- Возврат элемента:

return \$x

- Возврат последовательности:

return {\$x, \$c}

Возврат значений

- Возврат нового элемента

(будет создан элемент act с атрибутом fio, значение которого берут из переменной \$x/pers/@fio):

return <act fio={\$x/pers/@fio}>

- Возврат значения:

return data(\$x/pers)

Пример запроса

- Приведем пример запроса на языке XQuery на извлечение Ф.И.О актеров, имеющих более 2-х образований и не проживающих в Москве (здесь и далее запросы составлены к XML-документу БД киновыпусков, см. пример выше):

```
for $x in document("filename.xml")/db/actor
let $c = $x/count(edu)
where $c > 2 and $a/addr != "Москва"
return $x
```

Примеры циклов

- Приведем примеры простых циклов
for \$x in (2,3)
return \$x + 1
- Данный цикл возвращает последовательность (3,4).

for \$x in (2,3), \$y in (5, 10)
return { \$x* \$y }

- Здесь вложенный цикл, возвращает последовательность (10, 20, 15, 30).

Условия и операции

- Язык XQuery дополнительно поддерживает *операции*:
+, -, * , div, mod, sum(),
avg(), max(), min(), count()
- Для задания *условия* используют оператор **if –then – else**,
например:

```
return { if ( $x/@at= "ABC ") then <a1>{data($x/t2 )}< /a1 >  
         else <a1/> }
```

Сравнение узлов

- Сравнение *идентичности* узлов XML-документа осуществляют с помощью операторов **is**, **is not**, **<<**, например:

\$n1 is \$n2 — узлы \$n1 и \$n2 идентичны;

\$n1 is not \$ n2 — не идентичны;

\$1 << \$2 — n1 встречается ранее в документе, чем n2

Конструктор элементов

- Для генерации элементов «на лету» :
- из существующих узлов { значения }. Например, из переменной \$a взять атрибут id и вложенный элемент b:

{ \$a/@id, \$a/b }

- с указанием имени нового элемента
{element имя значение}
- Например для элемента из переменной \$e взять ее название и добавить все атрибуты, и ее содержимое, умноженное на 2:

element { name(\$e) } { \$e/@*, data(\$e) *2 }

Пример генерации на лету

- Пример создания элемента A с атрибутами id и n и вложенным элементом:

**element A { attribute id { 1}, attribute n {"name"},
 element B { 1234} }**

- результатом будет новый элемент:

1234

ФУНКЦИИ

```
declare function fun($var as xs:integer) as element(actor)*  
{  
  return <actor age={$var}/>  
}
```

- Эта функция принимает на вход целое число и возвращает ноль или более элементов actor.
- Для вызова функции указывают ее имя и фактические параметры, например:

```
return { fun(20) }
```

Пример запроса XQuery

- Запрос, который ищет актеров, снявшихся более чем в трех фильмах, и возвращает Ф.И.О актера, его 1-ое образование и количество его фильмов с сортировкой по Ф.И.О:

```
for $a in document( "filename.xml")//actor
let $f:= //film[ contains($a/@tof, @fid) ]
where count($f) >3
order by $a/fio
return { $a/fio, $a/edu[1], { count ($f ) } }
```

Пример запроса XQuery -2

- Запрос на формирование списка Ф.И.О всех актеров:

```
< ul >
{
  for $x in doc ( "filename.xml" )//fio/text ( )
  order by $x
  return <li>{ $x} </li>
}
< ul >
```

Спасибо за внимание!

Виноградова Мария Валерьевна

Vinogradova.m@bmstu.ru

МГТУ им. Н.Э. Баумана

кафедра Систем обработки информации и
управления (ИУ5)